

GG3: Neural Data Analysis

Dan Logan, DL725
Selwyn College

1. Introduction

In 2015, Latimer et al. challenged the widely accepted drift-diffusion model for macaque lateral intraparietal (LIP) spike trains recorded during a ‘random dot motion’ task. While the traditional model assumes single-trial firing rates continuously ramp up through evidence accumulation, Latimer proposed a stepping model where firing rates jump instantly from a minimum to a maximum rate at a specific step time. Whether decision-making occurs within the LIP remains debated, as a single step suggests the computation happens elsewhere.

This four-week project analyses the statistical properties of both models. The final report will apply Hidden Markov Models (HMMs) and the forward-backward algorithm to estimate hidden latents, while exploring robustness and model mismatch.

This week-one report outlines the foundational understanding and simulation of these models to reproduce experimental data. Firing rate trajectories are visualized and passed through a Poisson process to generate spike rasters. Trial-averaged behaviours are analysed using Peri-stimulus time histograms (PSTHs) and quantified using the Fano Factor. Finally, an ad hoc method for determining which model a dataset was generated by is developed and tested.

2. Investigation and Analysis

2.1 Visualising the models

Firing rate plots and spike rasters were plotted to show how each models firing rate changes over time, and the resulting effects on the spike trains. Figure 1 shows how the step model has an instantaneous increase in firing rate resulting in more dense spike locations after the step marker. Figure 2 displays the gradual ramping behaviour of the drift-diffusion model and a similar gradual increase in spike density.

The effects of parameters m and r have been displayed in figure 1 and figure 3, with m acting as the mean of the step positions. Both figures also show how parameter r controls the variance of the model, with variance increasing as r decreases for a given m .

For the ramping model, the effect of the drift constant β is shown in figure 2, with a larger β resulting in the firing rate reaching the maximum rate in a shorter amount of time. Sigma controls the diffusion of the model, and its effects are shown in figure 4. For a given β , a larger σ leads to a higher variance in the time it takes to reach the maximum firing rate.

2.2 Peri-stimulus time histograms

Peri-stimulus time histograms (PSTHs) were plotted to show the trial average firing rates for different parameter regimes. The subplots in figure 5 show that for the stepping model, parameter r controls how centralised the steps are on average around the mean m . The subplots of figure 6 show β controlling the average time of reaching the maximum firing rate, however, σ does not

significantly change the plot shape as the PSTH plots the average firing rate. The final subplot in both figures shows the effect of the number of trials used to create the PSTH, with fewer trials resulting in stronger fluctuations. This can cause issues as experimental trials are usually in the 100s where fluctuations are strong. Figure 7 shows parameter regimes for both models such that the PSTHs look near identical, thus proving analysis of PSTHs cannot be used alone to determine which model better describes the underlying process behind the spike trains.

2.3 Fano Factor

To extend beyond first order statistics, the Fano Factor, given by the variance over the mean in each bin, was plotted for both models with varying parameters. Figure 8 shows the step model producing single peaks located at the mean that get wider as parameter r drops. Figure 9 shows that in general the drift diffusion model starts at 1, initially climbs up, before gradually drops back down to 1. The drift constant β determines how early the hump is, and σ determines how high this hump is.

2.4 Model data classification

Different methods for building an ad hoc classifier for identifying which model produced a set of spike trains were tested. Smoothed PSTHs were plotted and the maximum gradients, and how well a sigmoid fit to the curve, were both tested as method of distinguishing between models. However, both suffered from the lack of sigmoid shapes in the step models PSTH due to low r values which decreases the concentration of step times around the mean. However, a low-frequency sinusoidal-wave trend was spotted in the Fano Factor graph which was more easily distinguishable compared to the drift diffusion graphs. DFTs were taken of the Fano Factor graphs, shown in figures 10 and 11, and the 1 Hz power was used to separate the two models. A test was conducted with 60 different data sets, each containing 400 trials and uniformly generated parameters across the parameter range. A threshold was set at a Power of 5.5 for 1 Hz, with values over being attributed to the step model. Step model data was correctly classified 83.3% of the time, while drift-diffusion data was correctly classified 90% of the time. To further develop this classifier, optimisation could be performed on the threshold value, as the difference in correct model classification percentages suggests the threshold could be increased.

3. Conclusions

In the first week of this project:

- visualisation tools have been developed for understanding how the firing rate of the models behave over time, and likely spike trains that could arise from these firing rates
- PSTHs have been plotted to show the time average firing rates of different parameter regimes, uncovering problems when both models produce similar PSTHs
- Plots of Fano Factor over time were explored as a way of incorporating the variance of firing rate paths into the analysis of the models
- An ad hoc, non-Bayesian method of classification using the DFT of the Fano Factor plots was developed to estimate which model a certain dataset came from, resulting in 90% of drift-diffusion model data being correctly identified, and 83.3% of step model data being correctly identified

4. Figures

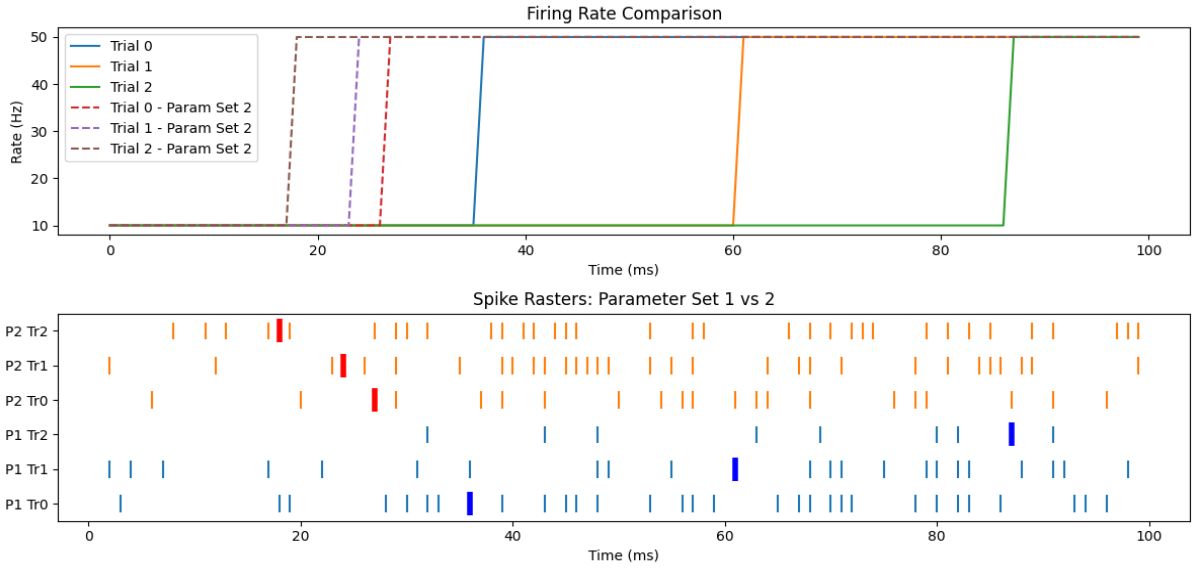


Figure 1: Firing rate and spike raster for step models with $m = 25$, $r = 10$ (dotted) and $m = 75$, $r = 10,000$ (filled)

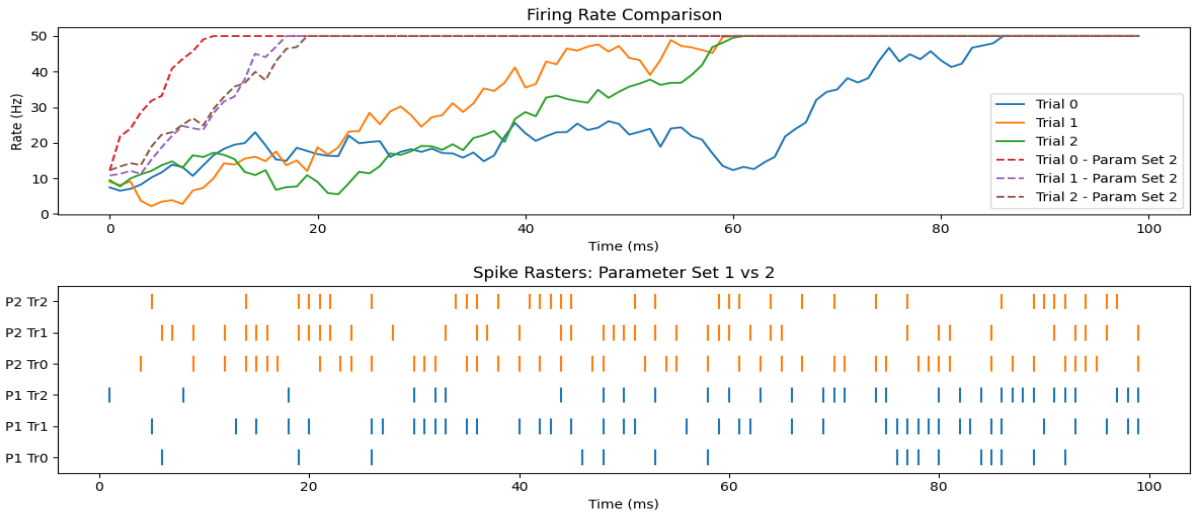


Figure 2: Firing rate and spike raster for drift-diffusion model with $\beta = 4$, $\sigma = 0.5$ (dotted) and $\beta = 0.8$, $\sigma = 0.5$ (filled)

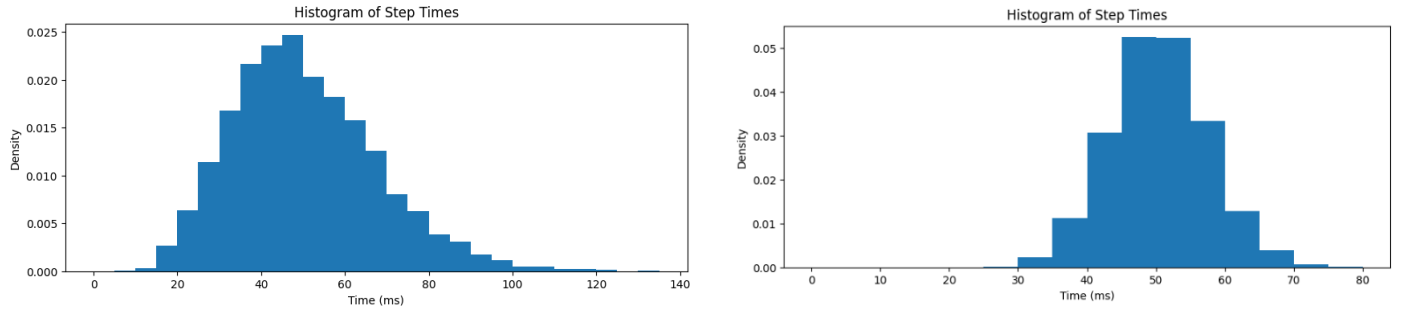


Figure 3: Histogram of step times for $r = 10$ (left) and $r = 10,000$ (right)

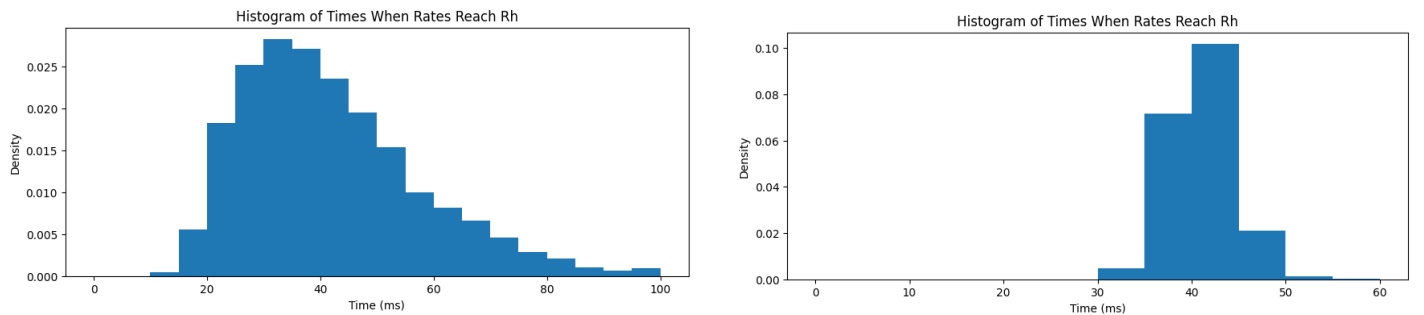
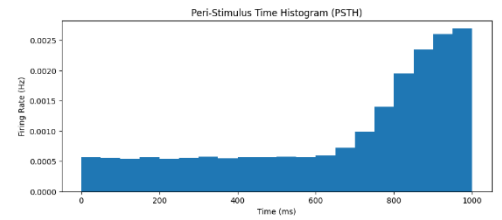
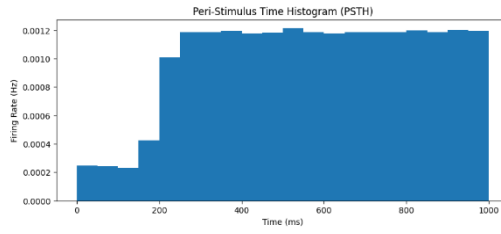
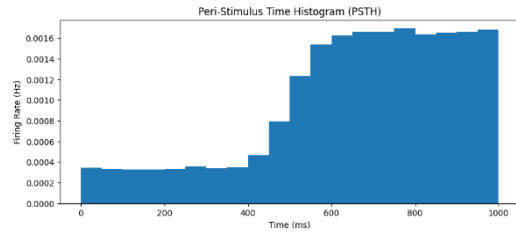


Figure 4: Histograms of time taken to reach maximum firing rate for $\text{SIGMA} = 0.5$ (left) and $\text{SIGMA} = 0.1$ (right)

Step Model PSTH: $m=500$, $r = 100$, No. trials = 5000

Step Model PSTH: $m=200$, $r = 100$

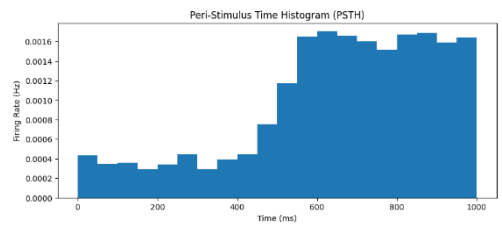
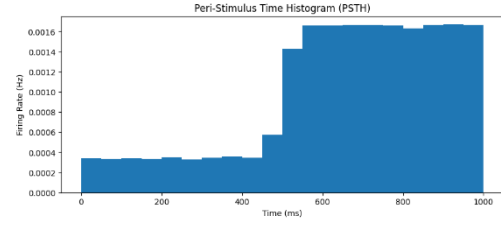
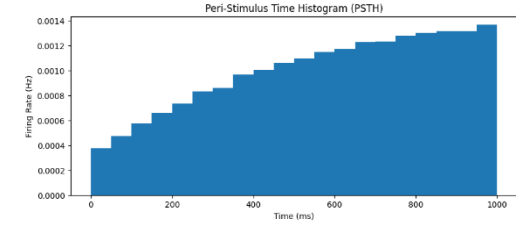
Step Model PSTH: $m=800$, $r = 100$



Step Model PSTH: $m=500$, $r = 1$

Step Model PSTH: $m=500$, $r = 10,000$

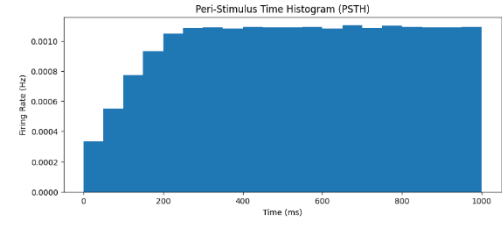
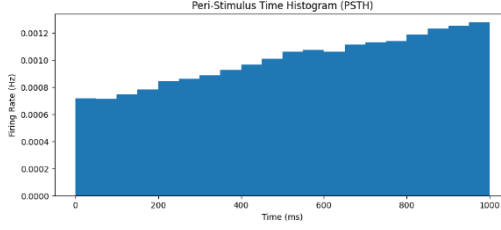
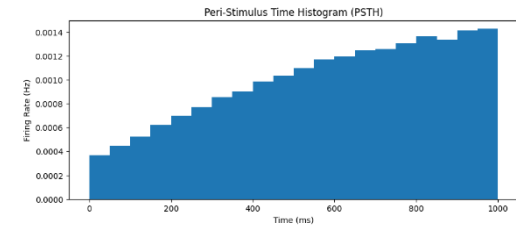
Step Model PSTH: No. trials = 100



D-D Model PSTH: $\beta = 1$, $\sigma = 0.5$, No. trials = 5000

D-D Model PSTH: $\beta = 0.1$, $\sigma = 0.5$

D-D Model PSTH: $\beta = 4$, $\sigma = 0.5$



D-D Model PSTH: $\beta = 1$, $\sigma = 0.01$

D-D Model PSTH: $\beta = 1$, $\sigma = 2$

D-D Model PSTH: No. trials = 100

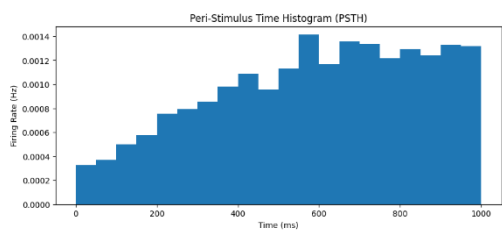
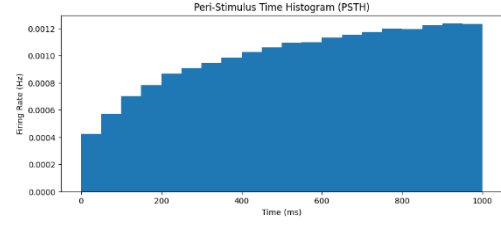
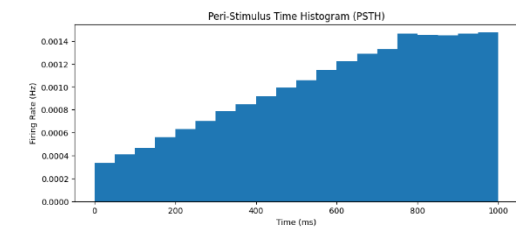


Figure 5: Subplots (top 6) showing PSTH of step model with varying parameters and no. trials

Figure 6: Subplots (bottom 6) showing PSTH of drift-diffusion model with varying parameters and no. trials

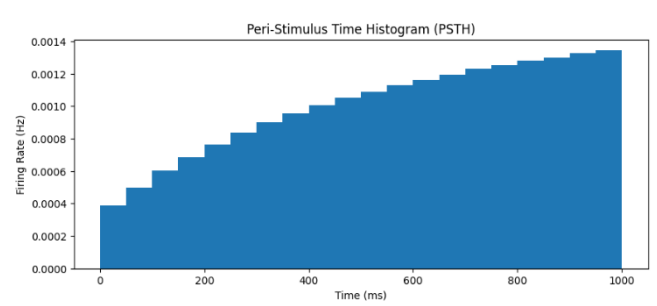
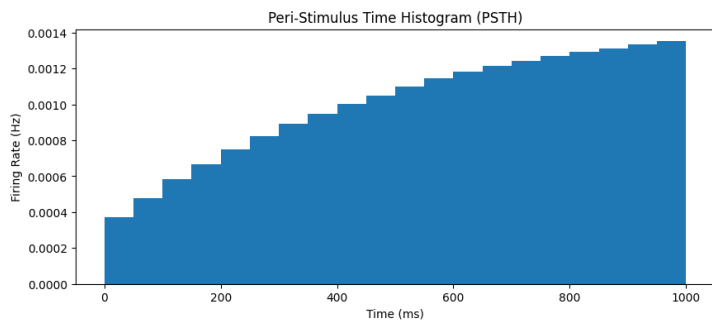


Figure 7: Step model PSTH for $m = 500$, $r = 1$ (left) and ramp model PSTH for $B = 1$, $SIGMA = 1$ displaying similar shapes

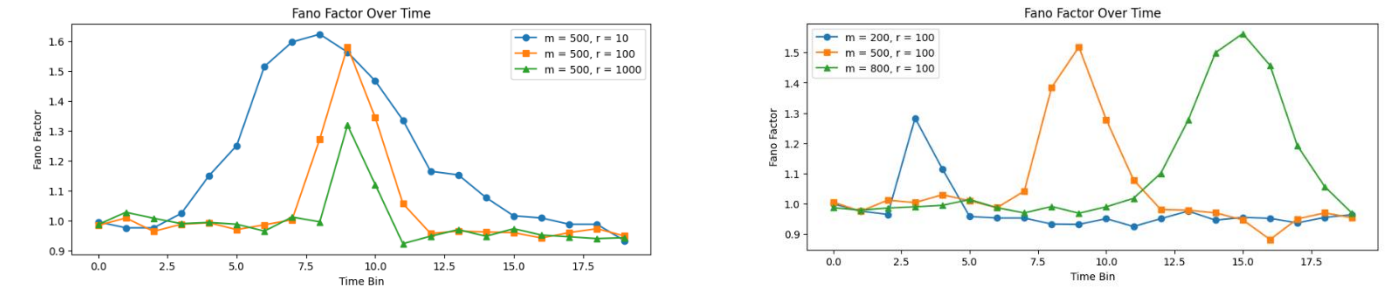


Figure 8: Step model Fano Factor for varying parameters

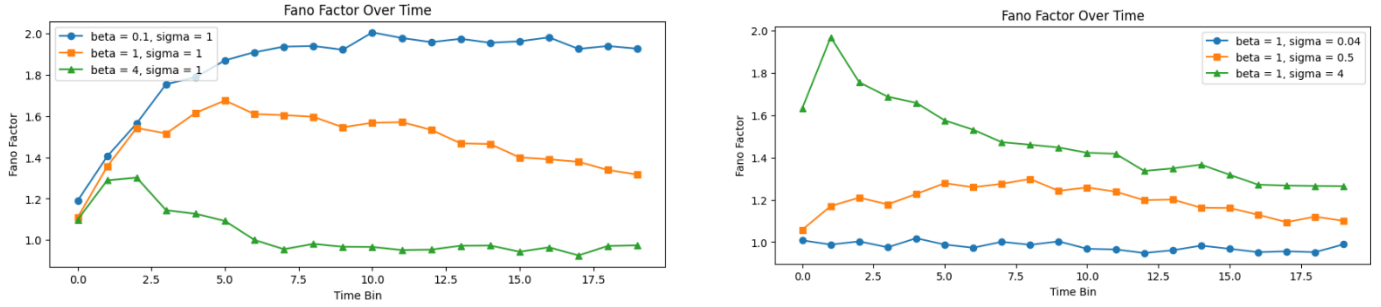


Figure 9: Drift-diffusion model Fano Factor for varying parameters

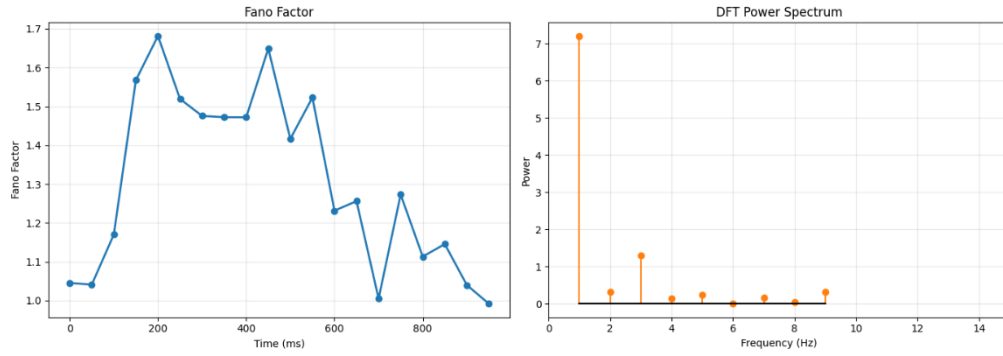


Figure 10: DFT of Fano Factor of step model

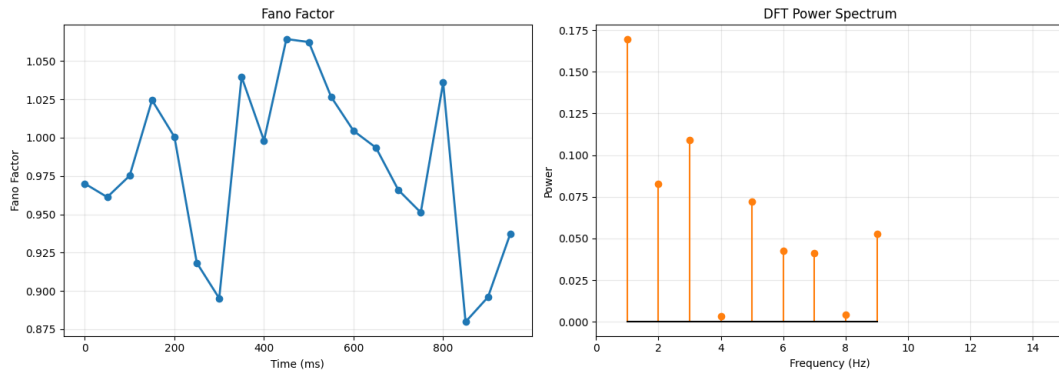


Figure 11: DFT of Fano Factor of drift-diffusion model

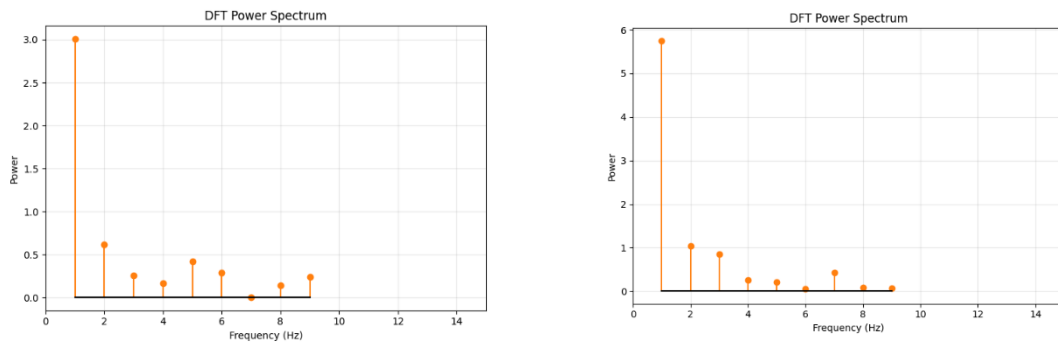


Figure 12: Examples of mis-classified parameter regimes, step model with $r = 0.5$ (left) and drift-diffusion model with $\text{SIGMA} = 3.7$ (right)

5. Appendix (Code)

-----Task 1.1-----

```
test_step_model = models.StepModel(m=50, r=10)
step_spikes, step_jumps, step_rates = test_step_model.simulate(Ntrials=5000)

test_2_step_model = models.StepModel(m=50, r=10000)
step_spikes2, step_jumps2, step_rates2 = test_2_step_model.simulate(Ntrials=5000)

Plotting_Dan.plot_spikes_and_rates(step_spikes, step_rates, 3, Step_Times = step_jumps,
multiparams=True, spikes2 = step_spikes2, rates2 = step_rates2, Step_Times2 = step_jumps2)
Plotting_Dan.histogram_step_times(step_jumps, binsize=5)
Plotting_Dan.histogram_step_times(step_jumps2, binsize=5)

test_ramp_model = models.RampModel(beta=2, sigma=0.1)
ramp_spikes, ramp_jumps, ramp_rates = test_ramp_model.simulate(Ntrials=5000)

test_2_ramp_model = models.RampModel(beta=2, sigma=0.5)
ramp_spikes2, ramp_jumps2, ramp_rates2 = test_2_ramp_model.simulate(Ntrials=5000)

Plotting_Dan.plot_spikes_and_rates(ramp_spikes, ramp_rates, 3, multiparams=True, spikes2 =
ramp_spikes2, rates2 = ramp_rates2)
Plotting_Dan.histogram_Rh_times(ramp_rates, binsize=5)
Plotting_Dan.histogram_Rh_times(ramp_rates2, binsize=5)
```

-----Task 1.2-----

```
psth_step_model = models.StepModel(m=500, r=3)
psth_step_spikes, psth_step_jumps, psth_step_rates =
psth_step_model.simulate(Ntrials=5000, T=1000)
Plotting_Dan.plot_PSTH(psth_step_spikes, Ntrials=400, binsize=50)

psth_ramp_model = models.RampModel(beta=1, sigma=0.04)
psth_ramp_spikes, psth_ramp_jumps, psth_ramp_rates =
psth_ramp_model.simulate(Ntrials=5000, T=1000)
Plotting_Dan.plot_PSTH(psth_ramp_spikes, Ntrials=400, binsize=50)
```

-----Task 1.3-----

```
fano_step_model = models.StepModel(m=500, r=0.5)
fano_step_spikes, fano_step_jumps, fano_step_rates = fano_step_model.simulate(Ntrials=400,
T = 1000)

fano_2_step_model = models.StepModel(m=500, r=2)
fano_step_spikes2, fano_step_jumps2, fano_step_rates2 =
fano_2_step_model.simulate(Ntrials=400, T = 1000)

fano_3_step_model = models.StepModel(m=500, r=6)
fano_step_spikes3, fano_step_jumps3, fano_step_rates3 =
fano_3_step_model.simulate(Ntrials=400, T = 1000)
```

```
Plotting_Dan.fano_factor(fano_step_spikes, T = 1000, binsize = 100, multifano=True,
spikes2=fano_step_spikes2, spikes3=fano_step_spikes3)
```

```
fano_ramp_model = models.RampModel(beta=1, sigma=0.1)
fano_ramp_spikes, fano_ramp_jumps, fano_ramp_rates = fano_ramp_model.simulate(Ntrials=400,
T = 1000)
```

```
fano_2_ramp_model = models.RampModel(beta=1, sigma=0.5)
fano_ramp_spikes2, fano_ramp_jumps2, fano_ramp_rates2 =
fano_2_ramp_model.simulate(Ntrials=400, T = 1000)
```

```
fano_3_ramp_model = models.RampModel(beta=0.1, sigma=4)
fano_ramp_spikes3, fano_ramp_jumps3, fano_ramp_rates3 =
fano_3_ramp_model.simulate(Ntrials=400, T = 1000)
```

```
Plotting_Dan.fano_factor(fano_ramp_spikes, T = 1000, binsize = 100, multifano=True,
spikes2=fano_ramp_spikes2, spikes3=fano_ramp_spikes3)
```

-----Task 1.4-----

```
random_m = np.random.uniform(250, 750)
random_r = np.random.uniform(0.5, 6)
print(random_m, random_r)
random_beta = np.random.uniform(0, 4)
random_sigma = np.random.uniform(0.04, 4)
print(random_beta, random_sigma)
adhoc_ramp_model = models.RampModel(beta=random_beta, sigma=random_sigma)
label_text = f"Ramp Model (b={random_beta:.1f}, s={random_sigma:.1f})"
```

```
adhoc_step_model = models.StepModel(m=random_m, r=random_r)
label_text = f"Step Model (m={random_m:.1f}, r={random_r:.1f})"
```

```
adhoc_ramp_spikes, adhoc_ramp_jumps, adhoc_ramp_rates =
adhoc_ramp_model.simulate(Ntrials=400, T=1000)
adhoc_step_spikes, adhoc_step_jumps, adhoc_step_rates =
adhoc_step_model.simulate(Ntrials=400, T=1000)
```

```
Plotting_Dan.analyze_fano_dft(adhoc_ramp_spikes, T=1000, binsize=50, label=label_text)
Plotting_Dan.analyze_fano_dft(adhoc_step_spikes, T=1000, binsize=50, label=label_text)
```

-----Plotting_Dan-----

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```

def plot_spikes_and_rates(spikes, rates, no_of_plotted_trials=1, Step_Times=None,
multiparams=False, spikes2=None, rates2=None, Step_Times2=None, histograms=False):

    # plot spike raster and rate for first trial

    spike_times = []

    for trial in range(no_of_plotted_trials):
        trial_spikes = np.where(spikes[trial] > 0)[0] # np.where returns a list of
indexes where condition spikes > 0 is true
        spike_times.append(trial_spikes)

    if multiparams and spikes2 is not None and rates2 is not None:
        spike_times2 = []
        for trial in range(no_of_plotted_trials):
            trial_spikes2 = np.where(spikes2[trial] > 0)[0]
            spike_times2.append(trial_spikes2)

    plt.figure(figsize=(12, 6))
    plt.subplot(2, 1, 1)
    for trial in range(no_of_plotted_trials):
        plt.plot(rates[trial], label=f'Trial {trial}')

    if multiparams and rates2 is not None:
        for trial in range(no_of_plotted_trials):
            plt.plot(rates2[trial], label=f'Trial {trial} - Param Set 2', linestyle='-
-')

        plt.title('Firing Rate Comparison')
    else:
        plt.title('Firing Rate')
    plt.xlabel('Time (ms)')
    plt.ylabel('Rate (Hz)')
    plt.legend()

    plt.subplot(2, 1, 2)

    if multiparams and spikes2 is not None:
        # 1. Total rows = 2 sources * trials per source
        total_rows = no_of_plotted_trials * 2
        offsets_all = np.arange(1, total_rows + 1)

        # 2. Split offsets: first half for Set 1, second half for Set 2
        offsets1 = offsets_all[:no_of_plotted_trials]
        offsets2 = offsets_all[no_of_plotted_trials:]

        # 3. Plot Parameter Set 1 (using a clean default blue color 'C0')
        plt.eventplot(spike_times, lineoffsets=offsets1, linelengths=0.5, colors='C0')
        if Step_Times is not None:
            for trial in range(no_of_plotted_trials):

```



```

        plt.scatter(Step_Times[trial], offsets1[trial], color='blue',
marker='|', s=300, linewidth=4, zorder=10)

    # 4. Plot Parameter Set 2 (using a clean default orange color 'C1')
    plt.eventplot(spike_times2, lineoffsets=offsets2, linelengths=0.5,
colors='C1')
    if Step_Times2 is not None:
        for trial in range(no_of_plotted_trials):
            plt.scatter(Step_Times2[trial], offsets2[trial], color='red',
marker='|', s=300, linewidth=4, zorder=10)

    # 5. Create distinct labels for the Y-ticks to show which is which
    y_labels = [f'P1 Tr{i}' for i in range(no_of_plotted_trials)] + [f'P2 Tr{i}'
for i in range(no_of_plotted_trials)]
    plt.yticks(offsets_all, y_labels)
    plt.ylim(0.5, total_rows + 0.5)
    plt.title('Spike Rasters: Parameter Set 1 vs 2')

else:
    # --- Fallback to your original single-parameter setup ---
    offsets = np.arange(1, no_of_plotted_trials + 1)
    plt.eventplot(spike_times, lineoffsets=offsets, linelengths=0.5)

    if Step_Times is not None:
        for trial in range(no_of_plotted_trials):
            plt.scatter(Step_Times[trial], offsets[trial], color='red',
marker='|', s=300, linewidth=4, zorder=10)

        plt.yticks(offsets, [f'Trial {i}' for i in range(no_of_plotted_trials)])
        plt.title('Spike Raster')

plt.xlabel('Time (ms)')
plt.tight_layout()
plt.show()

def histogram_step_times(Step_Times, binsize=10):

    # plot histogram of step times across trials

    plt.figure(figsize=(10, 4))
    plt.hist(Step_Times, bins=np.arange(0, max(Step_Times) + binsize, binsize),
density=True)
    plt.title('Histogram of Step Times')
    plt.xlabel('Time (ms)')
    plt.ylabel('Density')
    plt.show()

def histogram_Rh_times(rates, binsize=10):
    # plot histogram of times when rates reach Rh across trials

    Rh_times = []
    for trial in range(rates.shape[0]):

```

```

    Rh_time = np.where(rates[trial] >= np.max(rates[trial]))[0][0] # find first
time index where rate reaches max (Rh)
    Rh_times.append(Rh_time)

plt.figure(figsize=(10, 4))
plt.hist(Rh_times, bins=np.arange(0, max(Rh_times) + binsize, binsize),
density=True)
plt.title('Histogram of Times When Rates Reach Rh')
plt.xlabel('Time (ms)')
plt.ylabel('Density')
plt.show()

def plot_PSTH(spikes, Ntrials=10, binsize=10):

    #histogram spikes from multiple trials to understand average firing rates

    all_spike_times = []
    for trial in range(Ntrials):
        trial_spikes = np.where(spikes[trial] > 0)[0]
        all_spike_times.extend(trial_spikes)

    plt.figure(figsize=(10, 4))
    plt.hist(all_spike_times, bins=np.arange(0, max(all_spike_times) + binsize,
binsize), density=True)
    plt.title('Peri-Stimulus Time Histogram (PSTH)')
    plt.xlabel('Time (ms)')
    plt.ylabel('Firing Rate (Hz)')
    plt.show()

def fano_factor(spikes, T = 1000, binsize=10, multifano=False, spikes2=None,
spikes3=None):

    #calculate Fano factor to understand variability in spike counts across trials

    #find loactions of spikes
    #iterate through bins and trials
    # cout number of spikes in each bin for each trial
    # calculate mean and variance of spike counts across trials for each bin
    #Fano factor = variance / mean

    fano_factors = []
    fano_factors2 = []
    fano_factors3 = []

    for bin in range(int(T / binsize)):

        spike_counts = []
        for trial in range(spikes.shape[0]):
            trial_spikes = np.where(spikes[trial] > 0)[0]
            count = np.sum((trial_spikes >= bin * binsize) & (trial_spikes < (bin + 1)
* binsize))
            spike_counts.append(count)

```

```

mean_count = np.mean(spike_counts)
variance_count = np.var(spike_counts)
fano_factor = round((variance_count / mean_count if mean_count > 0 else 0), 3)
fano_factors.append(fano_factor)

if multifano:
    for bin in range(int(T / binsize)):
        spike_counts2 = []
        for trial in range(spikes2.shape[0]):
            trial_spikes2 = np.where(spikes2[trial] > 0)[0]
            count2 = np.sum((trial_spikes2 >= bin * binsize) & (trial_spikes2 <
(bin + 1) * binsize))
            spike_counts2.append(count2)

        mean_count2 = np.mean(spike_counts2)
        variance_count2 = np.var(spike_counts2)
        fano_factor2 = round((variance_count2 / mean_count2 if mean_count2 > 0
else 0), 3)
        fano_factors2.append(fano_factor2)

    for bin in range(int(T / binsize)):
        spike_counts3 = []
        for trial in range(spikes3.shape[0]):
            trial_spikes3 = np.where(spikes3[trial] > 0)[0]
            count3 = np.sum((trial_spikes3 >= bin * binsize) & (trial_spikes3 <
(bin + 1) * binsize))
            spike_counts3.append(count3)

        mean_count3 = np.mean(spike_counts3)
        variance_count3 = np.var(spike_counts3)
        fano_factor3 = round((variance_count3 / mean_count3 if mean_count3 > 0
else 0), 3)
        fano_factors3.append(fano_factor3)

#plot with legend if multifano is true, otherwise just plot one line

plt.figure(figsize=(10, 4))
plt.plot(fano_factors, marker='o', label='beta = 1, sigma = 0.04')
if multifano:
    plt.plot(fano_factors2, marker='s', label='beta = 1, sigma = 0.5')
    plt.plot(fano_factors3, marker='^', label='beta = 1, sigma = 4')
    plt.legend()
plt.title('Fano Factor Over Time')
plt.xlabel('Time Bin')
plt.ylabel('Fano Factor')

plt.show()

### Grad based classifier not used ####

```

```

def grad_based_classifier(spikes, T=1000, binsize=50, Ntrials=400, smoothing=0.5):

    all_spike_times = []
    for trial in range(Ntrials):
        trial_spikes = np.where(spikes[trial] > 0)[0]
        all_spike_times.extend(trial_spikes)

    plt.figure(figsize=(10, 4))

    ax = sns.histplot(all_spike_times, bins=np.arange(0, max(all_spike_times) +
binsize, binsize),
                      stat="density", kde=True, kde_kws={'bw_adjust': smoothing})

    if len(ax.lines) > 0:
        kde_line = ax.lines[0]
        x_line = kde_line.get_xdata() # Time points
        y_line = kde_line.get_ydata() # Firing rate density

        gradients = np.gradient(y_line, x_line)

        start_time = T / 4
        end_time = (3 * T) / 4

        in_range_indices = np.where((x_line >= start_time) & (x_line <= end_time))[0]

        if len(in_range_indices) > 0:
            gradients_in_range = gradients[in_range_indices]
            x_in_range = x_line[in_range_indices]

            max_grad = np.max(gradients_in_range)
            max_grad_idx = np.argmax(gradients_in_range)
            time_of_max_grad = x_in_range[max_grad_idx]

            y_of_max_grad = y_line[in_range_indices[max_grad_idx]]

            plt.scatter(time_of_max_grad, y_of_max_grad, color='red', s=100, zorder=5,
label=f'Max Slope: {max_grad:.6f} at
{time_of_max_grad:.1f}ms')
            plt.legend()

            print(f"Max Gradient in range [{start_time}ms, {end_time}ms]:
{max_grad:.6f} at {time_of_max_grad:.1f} ms")
        else:
            max_grad = None
            print("Specified time range was outside of the plotted data bounds.")
    else:
        max_grad = None
        print("Warning: Could not extract KDE line. Make sure data is non-empty.")

    plt.title('Spike Time Distribution with KDE & Max Gradient Spotting')
    plt.xlabel('Time (ms)')
    plt.ylabel('Density')
    plt.show()

```

```

if max_grad > 0.000005:
    print("Step Model")
else:
    print("Ramp Model")

# Return the value so you can pass it to your decision matrix logic
return max_grad

def analyze_fano_dft(spikes, T=1000, binsize=10, label="Model Data"):

    #calculates dft of fano factor graphs looking for low freq wave

    fano_factors = []
    num_bins = int(T / binsize)

    for bin_idx in range(num_bins):
        spike_counts = []
        for trial in range(spikes.shape[0]):
            trial_spikes = np.where(spikes[trial] > 0)[0]
            count = np.sum((trial_spikes >= bin_idx * binsize) & (trial_spikes <
(bin_idx + 1) * binsize))
            spike_counts.append(count)

        mean_count = np.mean(spike_counts)
        variance_count = np.var(spike_counts)
        ff = variance_count / mean_count if mean_count > 0 else 0
        fano_factors.append(ff)

    fano_factors = np.array(fano_factors)

    ### Compute DFT

    dt_seconds = binsize / 1000.0

    fft_coeffs = np.fft.fft(fano_factors)

    power_spectrum = np.abs(fft_coeffs) ** 2    # Calculate the Power Spectrum

    frequencies = np.fft.fftfreq(num_bins, d=dt_seconds)    # Get the corresponding
frequencies in Hz

    positive_indices = np.where(frequencies >= 0)[0]    # use only positive
frequencies
    frequencies = frequencies[positive_indices]
    power_spectrum = power_spectrum[positive_indices]

    plt.figure(figsize=(14, 5)) # Time-domain Fano Factor curve

    plt.subplot(1, 2, 1)

```

```

time_axis = np.arange(num_bins) * binsize
plt.plot(time_axis, fano_factors, marker='o', color='C0', linewidth=2)
plt.title(f'Fano Factor')
plt.xlabel('Time (ms)')
plt.ylabel('Fano Factor')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2) # Frequency-domain Power Spectrum

non_zero_idx = frequencies > 0 # Filter out the 0 Hz

plt.stem(frequencies[non_zero_idx], power_spectrum[non_zero_idx],
         linefmt='C1-', markerfmt='C1o', basefmt='k-')
plt.title('DFT Power Spectrum')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power')

plt.xlim(0, 15) # Zoom in on the low frequencies
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

freq_1hz_idx = np.argmin(np.abs(frequencies - 1.0))
power_at_1hz = power_spectrum[freq_1hz_idx]

if power_at_1hz > 5.5:
    print(f"Step Model")
else:
    print(f"Drift Diffusion Model")

```